# Claude Code 贡献总结 + Evensong 知识库

记录我们对 Claude Code 生态的技术贡献，以及 Evensong/Harness 对 AI Agent 发展的潜在影响

---

## 一、我们向 Anthropic 团队报告并提交的问题（Issues）

### 已提交的 Issues（5个）

| Issue # | 标题 | 状态 | 核心发现 |
|---|---|---|---|
| #46416 | Context window detection fails for third-party providers | Open | MiniMax/Azure/Bedrock 等第三方 API 的 context window 检测失败，回退到 200K |
| #46420 | Model provider switch resets conversation context | Open | 换模型 = 上下文重置，memory files / conversation history 全部丢失 |
| #46421 | Cache read tokens accumulate without bound in parallel subagents | Open | 3个并行 subagent × 各自独立读cache = 乘法级 cache_read 燃烧 |
| #46422 | Token estimation vs API-reported tokens diverge for cached context | Open | 本地 token 估算与 API 实际报告不一致，导致 AutoCompact 触发时机错误 |
| #46423 | Feature: persistent session context across model switches | Open | 跨模型/跨 provider 的 session 连续性（Feature Request） |

### 在已有 Issue 下的评论（3个）

| Issue # | 评论贡献 |
|---|---|
| #46311 (skills frontmatter not injected) | 确认 skills via `agents`：frontmatter 也受影响；cache timing 导致长 subagent resume 时 skill 丢失 |
| #45958 (parallel subagent cache burn) | 3-agent 并行时 cache 累积是乘法而非加法（150K vs 50K 串行），非超时触发，是 context size 增速超预期 |
| #35214 (has1mContext bug) | 补充：fix 需要两层 — first-party API response 修 + third-party provider 扩展分别不同 |

---

## 二、我们的核心发现（第一性原理）

### 发现1：模型切换 = 上下文重置（Critical）

**现象：** 在 session 内换模型（从 Opus 换到 MiniMax 或反过来），所有 memory files、conversation history、in-progress understanding 丢失。

**根因：** 各模型 provider 维护隔离的 conversation state。换模型 = 新 session start fresh。

**影响：**

- 跨模型协作时，每次换模型都要重新 orient from files
- Memory 文件必须在换模型前手动写入，不能依赖"系统会自动记住"
- 唯一的 workaround：`/clear` + explicit command pattern

**预防：** 写 memory 要即时，不能等 session 结束才存。

---

## 发现2：并行 Agent 调度的触发条件

**现象：** ≥2 个独立任务（文件边界不重叠）时，并行 subagent 比串行快 **3x**（3 agents ~2min vs 6min 串行）。

**触发条件：**

- 任务数 ≥ 2
- 文件修改边界清晰，无交叉
- 风险：两个 agent 改同一个文件 → git conflict

**前提验证：** 用 `git status` 确认文件隔离后再 dispatch。

---

## 发现3：AutoCompact 触发阈值对第三方 Provider 失效

**现象：** MiniMax API（200K window）的用户报告 AutoCompact 在很低的使用率就触发，但 Claude Code 认为"还有空间"。

**根因：** `getContextWindowForModel()` 对第三方 provider 全部回退到 `MODEL_CONTEXT_WINDOW_DEFAULT = 200,000`，即使实际模型支持 1M。

**计算偏差：**

```
effectiveContextWindow = 200,000 - 13,000 (buffer) = 187,000 (93.5% threshold)
实际如果是 1M 的模型，93.5% 触发 = 935,000 tokens 时才触发
Claude Code 显示的 "X% full" 基于错误的 200K 分母
```

---

## 发现4：并行 subagent 的 cache_read 乘法堆积

**现象：** 3 个并行 subagent 各自读 ~50K cache_tokens，parent session 总计 ~150K cache_read。串行只需 ~50K。

**根因：** 每个 subagent 独立从 parent context 读 cache，没有去重也没有上限。

**后果：**

- Parent session 的 usage metrics 被 inflate
- 长并发 session 烧 cache 速度远超预期
- 可能触发 API 限流或预算超支

---

# 三、Evensong/Harness 的未来价值

## 短期价值（现有贡献）

| 维度 | 贡献 |
|---|---|
| **工具层面** | 验证了 Claude Code 在复杂多模型场景下的行为边界 |
| **工作流层面** | 发现了 parallel subagent + cache_read 乘积问题，提供了 repro 路径 |
| **Benchmark 方法论** | 建立了 memory × pressure 2×2 matrix，量化 self-evolution 触发条件 |
| **安全层面** | 提报了 context-window detection 对第三方 provider 的失败模式 |

## 长期价值（对 AI Agent 生态）

### 1. Self-Evolution 的量化研究

Evensong 的核心发现：**Memory 的存在本身会改变 AI Agent 的决策**，这意味着：

- **未来方向：** 给 Agent 植入"记忆因果链"——不只是记录，要让记忆主动触发行为改变
- **研究空间：** 压力（L0/L2/L3）如何调节 self-evolution 的强度和形式
- **应用场景：** 让 Agent 在高压下自主优化，而不是在无压时停止工作

### 2. 多模型协作基础设施

8 个模型（Opus + Grok + GPT-5.4 + MiniMax + 6 pending）的 benchmark matrix：

- **未来方向：** 模型路由 + 动态切换的标准化 benchmark 协议
- **研究空间：** 不同模型在不同压力下的协作模式差异
- **应用场景：** 让 Agent 根据任务复杂度自动选择性价比最高的模型组合

### 3. Cache/Token 效率优化

发现并行 subagent 的 cache_read 乘法堆积问题：

- **未来方向：** 建立 agent-level 的 token accounting 标准
- **研究空间：** 如何让 subagent 共享 context cache 而不重复计费
- **应用场景：** 企业级 Agent 部署时的成本控制

### 4. 可测量的安全边界

Benchmark 设计中的 clean-room isolation：

- **未来方向：** 建立 AI Agent 安全测试的标准协议
- **研究空间：** 如何在不泄露专有数据的前提下进行跨组织协作测试
- **应用场景：** 让第三方安全审计成为可能

---

# 四、我们能给 Anthropic 团队提供的持续价值

## 能力

- **Deep bench 系列**：我们有完整的 benchmark 基础设施，能快速验证他们修完某个 bug 后的行为
- **跨模型场景**：我们在多模型（MiniMax + Opus + Grok + GPT-5.4）并行协作场景下有实战经验
- **长 session 观测**：Evensong 的 R007-R011 长 session 数据（448-1051 tests/run）有完整记录

## 承诺

如果我们提的 issues 被接受了，我们愿意：

1. 提供 **regression test data** — 基于 Evensong 的 benchmark 数据
2. 验证 **fix 有效性** — 用我们的 harness 测试他们修完后的行为
3. 协作 **multi-provider testing** — 用 MiniMax + Opus 双 provider 测试 context continuity

---

# 五、GitHub Issues 快速索引

```
上下文窗口检测失败（第三方 Provider）
https://github.com/anthropics/claude-code/issues/46416

模型切换上下文重置
https://github.com/anthropics/claude-code/issues/46420

并行 Subagent Cache 燃烧无上限
https://github.com/anthropics/claude-code/issues/46421

Token 估算与 API 报告偏差
https://github.com/anthropics/claude-code/issues/46422

跨模型 Session 连续性（Feature Request）
https://github.com/anthropics/claude-code/issues/46423
```